Stat 5200: Analysis of Designed Experiments

Jesse Wheeler

Introduction

Objectives and overview

- This material is based Chapters 13–14 from the textbook "A First Course in Design and Analysis of Experiments" (First Edition) (Oehlert, 2000).
- Chapter outcomes include:
  - You will be able to identify and design a block design.
  - You will be able to describe strengths and weaknesses of block-designs.
  - You will be able to create and analyze the most common types of block designs, including completely randomized blocks and Latin Squares.

Introduction: beyond the CRD

- Most recently, we have focused on the analysis of data using ANOVA models.
- We will now discuss specific experimental designs.
- Fortunately, the skills built for analyzing data will remain relevant.
- Most design principles build on the completely randomized design, but add additional structure to the randomization procedure.
- The additional structure is often practical and natural, but can also provide an analytic advantage by reducing variance among observations.

Introduction: beyond the CRD II

- Recall how a completely randomized design (CRD) is specified:

Randomized Complete Block Designs

Blocking

- While considered the "gold standard", CRDs are not always possible or practical.
- In this module, we introduce the idea of blocking.

Blocking: basic idea

- Choose a set of experimental units that are similar in some way to form a "block".
- Because things within a block are similar, the variance within-blocks is reduced.
- Blocks are often natural distinctions: units from a specific supplier, units spatially close in a field, or units taken during similar times.

The Randomized Complete Block Design

- There are many types of block designs. We will focus first on complete block designs.

Randomized Complete Block Designs (RCB)

- There are g treatments.
- There are r blocks.
- Each block receives exactly 1 of the g treatments, for a total of $N = rg$ experimental units (EUs).
- Within each block, the treatment assignment for each EU is randomized.

CRD vs RCB

- How does a randomized complete block design differ from a completely randomized design?
  - CRD: All N EUs can be randomly assigned to any treatment.
  - RCB: We first pick / identify r blocks, which consist of g EUs each. Then, within each block, we randomly assign the g treatments.

CRD vs RCB II

Example: batch-based blocks

Suppose you work at a bakery, and you have r different farms that supply you with eggs. You are interested in testing new baking times for your recipe: $T1$, $T2$, $T3$, $T4$.

CRD vs RCB III

Example: temporal blocks

Suppose you work in a laboratory, and you would like to test three different chemical catalysts: $C1$, $C2$, $C3$. Your analysis instrument must be recalibrated each morning, and measurements are timely and expensive, so you take three measurements per day over r days.

CRD vs RCB IV

Example: spatial blocks

Suppose you work at an artisanal creamery and age cheese wheels in a climate-controlled cave with r different vertical shelves. Because temperature and humidity vary from the floor to the ceiling, you treat each shelf as a block. You are testing four different brine washes: $W1$, $W2$, $W3$, $W4$.

Complex example: Mealybugs (Ex 13.1)

- This next example shows a blocked design, and helps us remember the definition and importance of identifying experimental units. From Module 1:
  - Experimental Units (EU) are the entities which a treatment will be applied.
  - Measurement Units are the individual or piece of experiment material you actually measure.

Complex example: Mealybugs (Ex 13.1) II

Mealybugs on cycads

Zookeepers wish to test three different treatments to combat the spread of mealybugs: water (a control), horticultural oil (a standard non-toxic pesticide), and fungal spores.

- Five infested cycads (plants) are moved to a testing area. Three branches are randomly selected from each.
- Each branch is randomly assigned to one of the treatment groups.
- On each branch, two 3cm $\times$ 3cm patches are identified.
- Mealybugs are counted on both patches, treated, and then recounted after 3 days.

Complex example: Mealybugs (Ex 13.1) III

	Plant				
	1	2	3	4	5
Water	-9	18	10	9	-6
	-6	5	9	0	13
Spores	-4	29	4	-2	11
	7	10	-1	6	-1
Oil	4	29	14	14	7
	11	36	16	18	15

Figure 1: Table 13.1 from Oehlert (2000).

Complex example: Mealybugs (Ex 13.1) IV

- Question: What type of design is this? What are the experimental units, and measurement units?

Common counter example

- A common mistake that is made is believing a study is a block design, when it is not.
- Below is a such an example. It is NOT a block design, even though there are natural "blocks".

NOT a block design: pig pens

A farmer would like to test the effect of 3 different feeds on pig weight. The farmer has r pens of pigs, each with 2 pigs. The farmer randomly assigns each of the pens with one of the three different feeds. They record the change in weight of the individual pigs.

Common counter example II

- Question: What type of design is this? What are the experimental units, and measurement units?

Block design: summary

- Blocks of EUs are often natural (time, space, batch, etc). However, natural grouping does *not* imply a block design was used (e.g., assigning feed to pig-pens).
- Block designs are carefully planned, not haphazard. That is, for each block, we randomly assign a EU in the block exactly one of the g treatments. The number of blocks is unconstrained.
- Block designs refer to how randomness is performed. We can still have complete treatment structures, like factorial treatments, within each block.

Block design: summary II

- When and why do we block, versus CRD?
  - We block to increase power and precision. This is done by decreasing the error variance, which is achieved by finding groups of units that are homogeneous.
  - The RCB is an effective design where there is a single source of extraneous variation in the response that we can identify ahead of time and use to partition the units into blocks.
  - Blocking is done at the time of randomization; you can't construct blocks after the experiment has been run.

How to analyze RCB designs

- The ANOVA models we have used are also useful for RCB; we don't have to "re-invent the wheel".
- The exact same models, assumptions, software, and code will be used to analyze data from an RCB; we just have to interpret things a little differently.

Model for RCB designs

The standard model for the ith treatment in the jth block will be:

$$y_{ij} = \mu + \alpha_i + \beta_j + \epsilon_{ij}, \quad \epsilon_{ij} \overset{iid}{\sim} N(0, \sigma^2)$$

In this model,

- i: the index for treatment level. Ranges from $1, \ldots, g$.
- j: the index for the block. Ranges from $1, \ldots, r$.
- In this model, there is no interaction effect. This is both because there is not enough data to estimate the effect, but also because we believe the blocks themselves to have little impact on the treatment effect.
- This is very similar to a analyzing a two-way factorial analysis without replication.

Model for RCB designs II

- Our Sum-of-Squares breakdown is:

$$SS_{Total} = SS_{Treatment} + SS_{Block} + SS_{Error},$$

  That is, to see the treatment effect, we first account for the block-effect, which is how we reduce the variance among measurements.
- The primary difference between unreplicated 2-way ANOVA: We do not care about the block effect, and we cannot perform inference because the blocks are not random.

Mealybugs example

- Consider the Mealy-bugs example. Below are the data, where the response for each patch has been averaged.

Table 1: Average change in mealybug after each treatment for each EU (columns are plant).

	1	2	3	4	5
Water	-7.5	11.5	9.5	4.5	3.5
Spores	1.5	19.5	1.5	2.0	5.0
Oil	7.5	32.5	15.0	16.0	11.0

Mealybugs example II

```
plants <- read.csv('mealybugs.csv')
str(plants)

'data.frame': 15 obs. of  3 variables:
 $ trt       : chr  "Water" "Spores" "Oil" "Water" ...
 $ plant     : int  1 1 1 2 2 2 3 3 3 4 ...
 $ avgchange: num  -7.5 1.5 7.5 11.5 19.5 32.5 9.5 1...
```

Mealybug example (continued)

```
mod <- aov(avgchange ~ factor(plant) + trt, data = plants)
summary(mod)

                Df Sum Sq Mean Sq F value  Pr(>F)
factor(plant)  4  686.4  171.60   9.681 0.00371 **
trt            2  432.0  216.02  12.187 0.00373 **
Residuals      8  141.8   17.73
---
Signif. codes:
0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Mealybug example (interpretation)

- In the results above, R outputs a F-value and P-value for the block factor (`plant`). Again, measurements are not randomly assigned to blocks, so we will ignore this output.
- More importantly, we see a significant effect for treatment. That is, there is fairly strong evidence for differences in mealybug counts between the treatments.
- We should continue to check model assumptions; doing so does not indicate there are violations of our key assumptions.
- We may be interested in what the estimated treatment means are, after accounting for variance among the blocks. This is easily accomplished with the `emmeans` package and function.

Mealybug example (interpretation) II

```r
library(emmeans)
# Get treatment average, after adjusting for block effect.
em_mod <- emmeans(mod, ~ trt)
```

```
 trt     emmean   SE df lower.CL upper.CL
 Oil       16.4 1.88  8  12.0582    20.74
 Spores     5.9 1.88  8   1.5582    10.24
 Water      4.3 1.88  8  -0.0418     8.64

Results are averaged over the levels of: plant
Confidence level used: 0.95
```

Mealybugs: further analysis

- Just like any of our previous examples, we may want to perform tests for sets of contrasts.
- A natural set of contrasts in this case may be: all pairwise comparisons, in which case, Tukey's adjustment is a good choice to control for multiple comparisons.

Mealybugs: further analysis II

```
pairs(em_mod)
```

```
 contrast       estimate   SE df t.ratio p.value
 Oil - Spores       10.5 2.66  8   3.943  0.0106
 Oil - Water        12.1 2.66  8   4.544  0.0047
 Spores - Water      1.6 2.66  8   0.601  0.8236

Results are averaged over the levels of: plant
P value adjustment: tukey method for comparing a family of 3 est
```

Unbalanced blocks (missingness)

- We have assumed a complete block design, meaning each block sees each treatment exactly once.
- In practice, it can be common to have missing data.
- In this case, missingness causes issues for a block-analysis in the same way as an unbalanced factorial analysis: our SS terms are no longer independent.
- Just as before, we need to consider the difference between Type I (sequential SS), Type II (Hierarchical), and Type III (Fully Adjusted). Any type works, but think carefully about what you are doing.

Is blocking worth it?

- Aside from sometimes being a natural thing to do, there are two primary consequences for blocking in the analysis:
  - Good: Blocking can result in increased precision, by grouping observations into similar blocks. This reduces σ^2, and increases statistical power if done correctly.
  - Bad: If we blocked, we have to account / estimate the block effect. This reduces the degrees of freedom available for estimating the treatment effect! This has a negative effect on statistical power.
- Thus, there is a competing impact in blocking. If the blocks don't actually differ from each other, blocking *will* cost us degrees of freedom without reducing the sum of squares, so it can actually be worse!

Is blocking worth it? II

- Whether or not the blocking was useful can be estimated. See Chapter 13.2.3 of Oehlert (2000) for details on how to compute this. Maybe not useful, since you need the results of an experiment to estimate how useful blocking in the experiment would be.
- **Summary:** only block if there is reason to believe that there are homogeneous groups of EUs. **We cannot block after obtaining the data!**

Latin Squares

Multiple Blocking Variables

- Just as we started with ANOVA with a single factor then moved to a factorial model with multiple factors, we can consider the case with multiple blocking factors.
- Designs with more than one blocking factor can be complex to set up, and generally require restrictions on the number of EUs.

Latin Square designs

- Latin Square is a design that blocks for two sources of variation.

Latin Squares

- We have g treatments that we would like to test.
- There are two variables we wish to block (e.g., time and space).
- We simultaneously perform a randomized complete block for each blocking variable, careful to avoid repeats.
- Each treatment occurs exactly once in each row and each column.
- This design requires exactly g^2 EUs.

Latin Square designs II

$g = 2$

A	B
B	A

$g = 3$

A	B	C
B	C	A
C	A	B

$g = 4$

A	B	C	D
B	C	D	A
C	D	A	B
D	A	B	C

Latin Square designs III

- If you ignore the row blocking factor, the LS design is a RCB for the column blocking factor.
- If you ignore the column blocking factor, the LS design is a RCB for the row blocking factor.
- Both row and column blocking factors are *balanced* simultaneously, and a Latin Square design is like simultaneously doing two RCB.

Latin Square designs IV

Strengths and Weaknesses: LS designs

Latin Squares allow blocking on two sources of variation, potentially reducing variance and improving precision. However, they do have drawbacks.

- Restricted to *exactly* g^2 EUs. This may be too few, or even too many.
- Generally, LS designs have few degrees of freedom for estimating error.
- It may be difficult to obtain units that block nicely on both sources of variation.

Randomizing the LS design

- Though the idea is simple, it's surprisingly challenging to designing a Latin Square (LS).
- **Goal:** we want to be able to pick one random LS design, out of all possibilities. It's not as simple as randomly assigning treatments.
- What we need is to randomly select a particular LS square, then randomly label the treatments.

Randomizing the LS design II

Randomizing an LS design: Fisher-Yates procedure

- For $g = 2$, randomly assign treatment labels.
- For $g = 3, 4, 5$, randomly select a "standard square"—the first row and column are in-order (See Appendix C)—then randomly permute all rows except the first, randomly permute all columns, and randomly assign treatments to the letters.
- For $g = 6$, select a standard square at random, randomly permute all rows and columns, and randomly assign the treatments to the letters.
- For $g \geq 7$, choose any LS square, randomly permute the rows and columns, and randomly assign treatments to the letters.

Randomizing the LS design III

- Rather than doing it manually, we can use existing packages to do this for us.

```
# install.packages("agricolae")
treatments <- c("ctrl", "trt1", "trt2")
agricolae::design.lsd(treatments, seed = 122333)$sketch
```

```
     [,1]   [,2]   [,3]
[1,] "trt2" "trt1" "ctrl"
[2,] "ctrl" "trt2" "trt1"
[3,] "trt1" "ctrl" "trt2"
```

Analyzing Latin Square Designs in R

- Just like the RCB design, we fit an additive model without interaction terms:

$$y_{ijk} = \mu + \alpha_i + \beta_j + \gamma_k + \epsilon_{ijk}$$

  i is treatment level, j is row-block number, k is column-block number

- Interestingly, we don't have all g^3 combinations of (i, j, k), only g^2 of them. The design is specifically constructed to make this possible.

Analyzing Latin Square Designs in R II

```r
# Ensure that blocking and treatment variables are factors!
mod <- aov(response ~ row + col + trt, data = latin_data)
summary(mod)
```

Source	SS	DF	MS	F
Rows	SS_{Rows}	$g-1$	SS_{Rows}/DF	
Columns	SS_{Cols}	$g-1$	SS_{Cols}/DF	
Treatments	SS_{Trt}	$g-1$	SS_{Trt}/DF	MS_{Trt}/MS_E
Error	SS_E	$(g-2)(g-1)$	SS_E/DF	

Analyzing Latin Square Designs in R III

- The output provides F-tests for rows, columns, and treatments.
- We ignore the test results for `row` and `col`, focusing solely on `trt`.
- To obtain treatment means adjusted for both blocking dimensions, we use `emmeans`.

Analyzing Latin Square Designs in R IV

```r
library(emmeans)

# Estimated marginal treatment means adjusted for row and col
em_mod <- emmeans(mod, ~ trt)
summary(em_mod)

# Tukey-adjusted pairwise comparisons
pairs(em_mod)
contrast(em_mod, method = ...)
```

Crossover Designs

- Crossover designs are highly specific designs, but they are common enough to warrant discussion.
- They are a specific type of Latin Square design, and arises often in bio medical settings.
- Idea: We need to block on subjects (because each subject responds differently), and we also need to block on time-period (because there may be differences over time due to growth, aging, disease progression, etc).

Crossover Designs II

The crossover design

- The crossover is a specific type of Latin Square design, with the "row block" corresponding to patient / subject, and the "column block" corresponding to time.
- There are g treatments, g subjects, given over g time periods.

LS (crossover) example: drug delivery

Blood Concentration of Drugs

Blood concentration of drugs changes over time. It will start at zero, increase to some maximum level, then decrease to zero as the drug is fully metabolized.

- We wish to compare three methods for delivering a drug: a solution, a tablet, and a capsule. We anticipate large subject to subject differences, so we block on subject.
- There are three subjects, and each subject will be given the drug three times, once with each of the three methods.
- Each treatment method will be used once in the first period, once in the second period, and once in the third period.

LS (crossover) example: drug delivery II

		Subject 1		Subject 2		Subject 3	
Period	1	A	1799	C	2075	B	1396
	2	C	1846	B	1156	A	868
	3	B	2147	A	1777	C	2291

Figure 2: Area under the concentration curve for the three subjects and treatments. Screenshot of Table 13.2 from Oehlert (2000).

LS (crossover) example: drug delivery III

```
drugTime <- read.csv('drugTime.csv')
mod <- aov(
  area ~ factor(subect) + factor(period) + factor(trt),
  data = drugTime
)
summary(mod)
```

```
                Df Sum Sq Mean Sq F value  Pr(>F)
factor(subect)   2 261115  130557   29.05 0.03328 *
factor(period)   2 928006  464003  103.23 0.00959 **
factor(trt)      2 608891  304445   67.73 0.01455 *
Residuals        2   8990    4495
---
Signif. codes:
0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

drug delivery (continued)

```
library(emmeans)
em_mod <- emmeans(mod, ~trt)
contrast(em_mod, method = 'pairwise') # Could plot if wanted.
```

```
 contrast    estimate   SE df t.ratio p.value
 trt1 - trt2      -85 54.7  2  -1.553  0.4303
 trt1 - trt3     -589 54.7  2 -10.766  0.0155
 trt2 - trt3     -504 54.7  2  -9.213  0.0211

Results are averaged over the levels of: subect, period
P value adjustment: tukey method for comparing a family of 3 est
```

Additional Topics

Replicated LS designs

- With the RCB design, you can have more EUs (replicates) by selecting more blocks. With LS designs, the number of EUs is fixed at g^2.
- The ready extension of this is a replicated LS design. The idea: perform multiple LS designs.
- There are multiple ways this could be done:
  - Reuse row blocks
  - Reuse column blocks
  - Reuse neither, or both.

Replicated LS designs II

- *Reusability* depends on experiment and logistics.
  - Row or column blocks may correspond to different equipment used to obtain measurements, which could be reused more than once.
  - Conversely, a block may correspond to a specific batch, field, or time period, in which case the blocks may not be reused.
- From an analysis point of view, the advantage of reusing a block factor is that we will have more degrees of freedom for error.
- The risk when reusing a block factor is that the block effects may change change, and the assumption of constant block effects is invalid.

Replicated LS designs III

CO emmisions (Example 13.8)

Carbon monoxide (CO) emissions from automobiles can be influenced by the formulation of the gasoline that is used. We wish to test $g = 4$ different formulations on CO2 emissions.

- The impact of formulation likely depends on cars (Block 1), and routes (Block 2).
- Standard LS: pick four cars, four routes.
- Replicated LS: reuse the 4 cars on 12 routes (3 Latin-Squares, replicated rows); or have 12 cars use them 4 at a time in same 4 routes (replicated columns); or have 12 cars, 12 routes (no replicates).
- Replicates can repeat the same designs, or each come from uniquely designed squares.

Models for replicated LS designs

- Replicated LS designs have the potential to add a "square" effect. That is, the three replicated squares may have slightly different behaviors.
- They can also introduce "nested" effects: the specific effect of a block depends on the specific square (e.g., if car 5 was only used in the 2nd square, then we can't estimate the effect of that block using data from the 1st and 3rd squares).

Models for replicated LS designs II

Model for Replicated LS (Replicated Columns)

Consider a study measuring the CO_2 emissions of 4 different types of gasoline. We have 12 cars available for testing.

We suspect the specific car and the route it takes both impact CO_2 emissions.

Our design: Identify 4 representative routes. We will use a replicated Latin Square where columns (routes) are reused, but rows (cars) are not. We divide the 12 cars into 3 batches of 4. For each batch (a single Latin Square), we assign the 4 gasoline types so that each gasoline is tested exactly once per car and exactly once per route. We repeat this for all 3 batches.

Models for replicated LS designs III

- The "standard" model for these data is:

$$y_{ijkl} = \mu + \alpha_i + \beta_j + (\beta\delta)_{jl} + \gamma_{k(l)} + \delta_l + \epsilon_{ijkl}$$

  - α_i: Treatment effect (gasoline type), $i \in \{1, 2, 3, 4\}$.
  - β_j: Main Column effect (route), $j \in \{1, 2, 3, 4\}$. This factor is reused.
  - δ_l: Square effect (batch of cars), $l \in \{1, 2, 3\}$.
  - $(\beta\delta)_{jl}$: Route $\times$ Square interaction. Tests if the route effect changes across different squares.
  - $\gamma_{k(l)}$: Nested Row effect (car within square). Cars are unique to each square, so they do not cross with the square.
  - ϵ_{ijkl}: Residual error.

Analyzing in R

- We need to be careful about analyzing replicated squares in R, because the "correct" approach depends on how the data are encoded.
- Suppose we reused columns (routes), but not rows (cars). Then, our data *could* have cars encoded as 1–4, (with three replicates each, one for each LS), or encoded as 1–12 (unique car numbers, since they were unused).
- If unique numbers for unreplicated blocks (1–12):

```
aov(y ~ square*column + row + trt, data = ...)
aov(CO ~ square*route + car + gas, data = ...)
```

Analyzing in R II

- **If repeated numbers for unreplicated blocks (1–4)**:
- We must explicitly tell R that car 1 in square 1 is NOT the same vehicle as car 1 in square 2. We use the nesting operator (/) or the interaction operator (:) to create the $\gamma_{k(l)}$ term.

```
aov(y ~ square*column + square/row + trt, data = ...)
# OR
aov(y ~ column*square + square:row + trt, data = ...)
aov(CO ~ route*square + square:car + gas, data = ...)
```

Further Reading and Topics

- To extend the idea of blocking to 3 extraneous sources of variance, we extend to a Graeco-Latin Square design (Chapter 13.4)
- When there are more than g units in each block—specifically, multiples of g—we can have a Generalized RCB design; these we can analyze just like a two-way ANOVA with interactions.
- Finally, incomplete block designs are cases are specifically chosen when we can't get g units per block, but have $k < g$ units. Chapter 14 deals with specific types of designs that cleverly accommodate this situation.

References and Acknowledgements

Oehlert GW (2000). *A First Course in Design and Analysis of Experiments*. W. H. Freeman. ISBN 0-7167-3510-5.

- Compiled on November 10, 2026 using R version 4.5.2.
- Licensed under the Creative Commons Attribution-NonCommercial license. Please share and remix non-commercially, mentioning its origin.
- We acknowledge students and instructors for previous versions of this course / slides.